

DESENVOLVIMENTO GUI COM TKINTER

Apostila de Programação e Design Visual em Python

Guia Mestre Prático e Completo

Exemplos de Código, Gerenciadores de Layout e Projetos

2026

1. Introdução ao Tkinter

O **Tkinter** é a biblioteca padrão do Python para o desenvolvimento de Interfaces Gráficas de Usuário (GUI). Baseado no toolkit Tcl/Tk, o Tkinter oferece uma forma robusta, multiplataforma e extremamente eficiente para criar aplicações de desktop profissionais sem a necessidade de instalar dependências externas.

Conceito Chave: Uma aplicação GUI funciona de forma diferente de um script tradicional de terminal. Em vez de uma execução linear de cima para baixo, ela é baseada em **Eventos**. O programa renderiza os componentes e entra em um estado de espera ativa (o mainloop), aguardando interações do utilizador (cliques, digitação, movimentos do rato).

O Esqueleto Humilde de uma Aplicação

Para inicializar qualquer aplicação gráfica no Tkinter, precisamos obrigatoriamente de uma janela raiz (root) e da chamada do loop principal de eventos.

```
import tkinter as tk

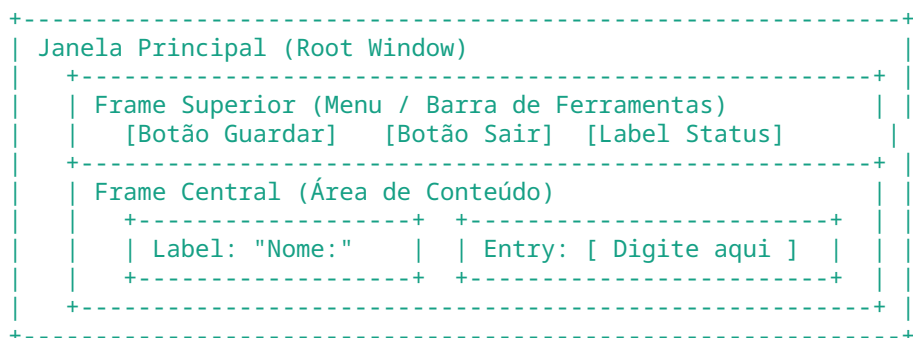
# 1. Criação da janela principal (Contentor Raiz)
root = tk.Tk()
root.title("Minha Aplicação Inicial")
root.geometry("450x300") # Definição de Dimensões (Largura x Altura)

# 2. Execução do ciclo de vida da aplicação
root.mainloop()
```

2. Desenho Arquitetural da Janela (Hierarquia)

O Tkinter organiza os elementos de forma hierárquica (estrutura em árvore). Os componentes gráficos são chamados de **Widgets** e precisam sempre de um elemento "pai" (master) que os contenha.

Estrutura Conceitual de Camadas



3. Gerenciadores de Layout (O Desenho da Tela)

Para que os widgets apareçam no ecrã, eles precisam de ser geridos por um organizador de espaço. O Tkinter fornece três mecanismos principais:

A) O Empacotador: `pack()`

Ideal para layouts lineares simples (estruturas verticais ou horizontais rápidas). Ele empilha os elementos consecutivamente.

```
import tkinter as tk
root = tk.Tk()
root.geometry("200x150")

# O parâmetro 'fill' expande o componente e 'pady' cria margem externa
btn_alto = tk.Button(root, text="Botão Superior", bg="#3498db", fg="white")
btn_alto.pack(side="top", fill="x", padx=10, pady=5)

btn_baixo = tk.Button(root, text="Botão Inferior", bg="#2ecc71", fg="white")
btn_baixo.pack(side="bottom", fill="x", padx=10, pady=5)

root.mainloop()
```

B) A Grelha / Matriz: `grid()`

O gerenciador mais poderoso e utilizado. Divide o espaço de forma invisível numa tabela estruturada por Linhas (row) e Colunas (column).

Mapeamento de Coordenadas Grid

	Coluna 0	Coluna 1
Linha 0	row=0, column=0	row=0, column=1
Linha 1	row=1, column=0	row=1, column=1

```
import tkinter as tk
root = tk.Tk()
root.title("Exemplo Grid")

# Construção de um formulário limpo
tk.Label(root, text="Utilizador:").grid(row=0, column=0, padx=10, pady=10, sticky="w")
tk.Entry(root).grid(row=0, column=1, padx=10, pady=10)

tk.Label(root, text="Palavra-passe:").grid(row=1, column=0, padx=10, pady=10, sticky="w")
tk.Entry(root, show="*").grid(row=1, column=1, padx=10, pady=10)

root.mainloop()
```

C) Posicionamento Absoluto: place()

Permite colocar um elemento explicitamente nas coordenadas exatas de pixel (**X**, **Y**). Não é recomendado para interfaces responsivas, mas oferece precisão total.

```
# O widget ficará fixo a 50px da esquerda e 80px do topo
botao_fixo = tk.Button(root, text="Ponto Fixo")
botao_fixo.place(x=50, y=80, width=120, height=35)
```

4. Catálogo Essencial de Componentes (Widgets)

Widget	Descrição Funcional	Exemplo de Sintaxe
Label	Exibe textos estáticos ou imagens na interface.	<code>tk.Label(pai, text="Olá")</code>
Button	Aciona funções executáveis ao ser clicado.	<code>tk.Button(pai, text="Ir", command=funcao)</code>
Entry	Campo de captura de texto de uma única linha.	<code>tk.Entry(pai, width=20)</code>
Text	Área rica para manipulação de textos multilinha.	<code>tk.Text(pai, height=5)</code>
Checkbutton	Caixa de seleção booleana (Ativado/Desativado).	<code>tk.Checkbutton(pai, text="Aceito")</code>
Listbox	Apresenta uma lista de opções selecionáveis.	<code>tk.Listbox(pai)</code>

Implementação Integrada de Componentes

```
import tkinter as tk
from tkinter import messagebox

def processar_dados():
    nome = campo_entrada.get()
    aceitou = var_check.get()
    if aceitou:
        messagebox.showinfo("Sucesso", f"Registro de {nome} Concluído!")
    else:
        messagebox.showwarning("Erro", "Tem de aceitar os termos de uso.")

root = tk.Tk()
root.title("Painel de Componentes")
root.geometry("380x300")

# Input de Texto
tk.Label(root, text="Nome Completo:", font=("Arial", 10)).pack(pady=5)
campo_entrada = tk.Entry(root, width=35)
campo_entrada.pack(pady=5)

# Checkbutton de Opção
var_check = tk.BooleanVar()
chk_termos = tk.Checkbutton(root, text="Declaro que li e aceito as condições",
                             variable=var_check)
chk_termos.pack(pady=10)

# Botão de Ação
btn_submeter = tk.Button(root, text="Submeter Cadastro", bg="#e67e22", fg="white",
                           command=processar_dados)
btn_submeter.pack(pady=10)

root.mainloop()
```

5. Organização Avançada com Frame

Para criar telas modulares e profissionais, utilizamos o componente Frame. Ele serve como um contentor lógico invisível, agrupando conjuntos específicos de widgets e isolando os seus gerenciadores de layout.

```

import tkinter as tk

root = tk.Tk()
root.geometry("500x350")
root.title("Arquitetura com Multi-Frames")

# --- FRAME SUPERIOR (Área de Título) ---
frame_topo = tk.Frame(root, bg="#2c3e50", height=60)
frame_topo.pack(fill="x", side="top")

lbl_titulo = tk.Label(frame_topo, text="SISTEMA ERP INTEGRADO", fg="white", bg="#2c3e50",
font=("Arial", 12, "bold"))
lbl_titulo.pack(pady=15)

# --- FRAME LATERAL DE MENU ---
frame_lateral = tk.Frame(root, bg="#34495e", width=120)
frame_lateral.pack(fill="y", side="left")

tk.Button(frame_lateral, text="Dashboard", width=12).pack(pady=10, padx=5)
tk.Button(frame_lateral, text="Clientes", width=12).pack(pady=10, padx=5)
tk.Button(frame_lateral, text="Definições", width=12).pack(pady=10, padx=5)

# --- AREA TRABALHO CENTRAL ---
frame_conteudo = tk.Frame(root, bd=2, relief="solid", bg="white")
frame_conteudo.pack(fill="both", expand=True, side="right", padx=10, pady=10)

tk.Label(frame_conteudo, text="Área de Trabalho Principal", font=("Arial", 11),
bg="white").pack(pady=50)

root.mainloop()

```

6. Projeto Prático Completo: Gestor de Tarefas Profissional

O exemplo abaixo unifica os conceitos de gerenciamento de estado (Variables), Listbox dinâmico, Scrollbars e caixas de diálogo nativas.

```

import tkinter as tk
from tkinter import messagebox

class AppTarefas:
    def __init__(self, root):
        self.root = root
        self.root.title("Gestor de Tarefas Avançado")
        self.root.geometry("420x450")
        self.root.config(bg="#f5f6fa")

        # Container Principal de Entrada
        self.frame_input = tk.Frame(self.root, bg="#f5f6fa")
        self.frame_input.pack(pady=15)

        self.txt_tarefa = tk.Entry(self.frame_input, font=("Arial", 12), width=24)
        self.txt_tarefa.pack(side="left", padx=5)
        self.txt_tarefa.bind("<Return>", lambda event: self.adicionar())

        self.btn_add = tk.Button(self.frame_input, text="Adicionar", bg="#2bc4aa",
fg="white", font=("Arial", 10, "bold"), command=self.adicionar)
        self.btn_add.pack(side="left")

        # Container da Listbox e Scrollbar
        self.frame_lista = tk.Frame(self.root)
        self.frame_lista.pack(pady=10)

        self.scroll = tk.Scrollbar(self.frame_lista)
        self.scroll.pack(side="right", fill="y")

        self.lista = tk.Listbox(self.frame_lista, font=("Arial", 11), width=35, height=12,
yscrollcommand=self.scroll.set, selectbackground="#3498db")
        self.lista.pack(side="left")
        self.scroll.config(command=self.lista.yview)

        # Botão de Remoção
        self.btn_remove = tk.Button(self.root, text="Remover Tarefa Selecionada",
bg="#e74c3c", fg="white", font=("Arial", 10, "bold"), command=self.remove)
        self.btn_remove.pack(pady=15)

    def adicionar(self):
        item = self.txt_tarefa.get().strip()
        if item:
            self.lista.insert(tk.END, item)
            self.txt_tarefa.delete(0, tk.END)
        else:
            messagebox.showwarning("Alerta", "O campo de texto não pode estar vazio.")

    def remove(self):
        try:
            indice = self.lista.curselection()[0]

```



```
        self.lista.delete(indice)
    except IndexError:
        messagebox.showwarning("Erro de Seleção", "Por favor, selecione uma tarefa da
lista.")

if __name__ == "__main__":
    janela = tk.Tk()
    app = AppTarefas(janela)
    janela.mainloop()
```